

Discord Casino Bot — Startup & Deployment Roadmap

Virtual-currency Discord casino/economy bot • MVP → SaaS → Deployment

Important: This roadmap assumes virtual, non-redeemable coins. Real-money betting, deposits, withdrawals, cash prizes, crypto, or purchasable betting currency require separate legal, payment, age/identity, jurisdiction, and Discord-policy review.

1. Product Vision

Build a Discord bot that turns a server into a social casino-style economy using virtual coins. Start small with a few games and an economy system, then add a web dashboard, analytics, tournaments, and premium features.

Core loop: Server owner installs bot → users receive virtual coins → users play games → balances update → leaderboards and rewards encourage engagement → server owners can configure and analyze the system.

2. MVP Features

Category	MVP
Economy	/balance, /daily, /profile, /leaderboard
Games	Coin Flip, Dice, Slots
Admin	Casino setup, settings, reset
Later	Blackjack, Roulette, tournaments, achievements, inventory, VIP, analytics

3. Recommended Tech Stack

Bot: Node.js + TypeScript + discord.js

Database: PostgreSQL + Prisma

Dashboard: Next.js + Tailwind CSS

Authentication: Discord OAuth2

Deployment: Managed Node.js hosting initially; move to VPS/AWS as scale requires.

4. Project Structure

```
discord-casino/  
├── src/  
│   ├── index.ts  
│   ├── commands/ (balance, daily, coinflip, dice, slots)  
│   ├── games/ (game logic)  
│   ├── services/ (economy, games)  
│   └── database/ (Prisma client)  
├── prisma/schema.prisma  
├── .env  
├── .gitignore  
└── package.json
```

5. Step-by-Step Build Process

1. Create Discord Application — Use the Discord Developer Portal, create an application, then add a bot. Keep the bot token secret.

- 2. Initialize Project** — Create a Node.js/TypeScript project and install discord.js, dotenv, Prisma, @prisma/client, TypeScript, tsx and Node types.
- 3. Connect Discord** — Initialize the Discord client and request only the permissions/scopes the bot actually needs.
- 4. Design Database** — Create models for User, Server, ServerMember, Wallet and Transaction. Keep server-specific balances.
- 5. Build Economy** — Implement starting coins, balance checking, daily rewards and safe balance updates.
- 6. Build Transactions** — Record every wager/reward with type, amount, before/after balance, game and timestamp. Use database transactions for atomic updates.
- 7. Build Coin Flip** — Validate bet → check balance → generate result server-side → calculate payout → update wallet → save transaction.
- 8. Build Dice** — Follow the same service pattern; validate inputs and prevent negative/oversized bets.
- 9. Build Slots** — Create symbols and a server-side payout table. Keep game results and odds server-side.
- 10. Add Protection** — Implement cooldowns, input validation, balance checks, rate limiting and protection against race-condition exploits.
- 11. Add Social Features** — Leaderboards, biggest wins, win streaks, most games played and weekly rankings.
- 12. Add Daily Rewards** — Create daily rewards and streak bonuses to encourage repeat engagement.
- 13. Build Dashboard** — Use Discord OAuth2 so admins can log in and configure their server.
- 14. Add Analytics** — Show members, active players, games played, coins wagered, game popularity and other server metrics.
- 15. Add Premium** — Offer advanced configuration, custom currency, analytics, tournaments, themes and other server-owner features.
- 16. Test Locally** — Use a private Discord server. Test normal behavior and intentionally try invalid bets, spam, simultaneous requests, restarts and database failures.
- 17. Deploy Database** — Use managed PostgreSQL and apply production Prisma migrations.
- 18. Deploy Bot** — Deploy the Node.js process with environment variables such as DISCORD_TOKEN, CLIENT_ID and DATABASE_URL.
- 19. Register Commands** — Use guild-specific commands during development; switch to production/global registration only after commands stabilize.
- 20. Create Installation Link** — Configure Discord installation with the required bot and application-command scopes and minimum necessary permissions.
- 21. Beta Launch** — Start with about five test servers. Offer early users free access in exchange for feedback.
- 22. Public Launch** — Create a landing page, Discord support server, bot installation flow and product documentation.
- 23. Iterate** — Add features based on real usage rather than building every game before launch.

6. Suggested Database Entities

User: id, discordId, username, createdAt

Server: id, discordGuildId, settings

ServerMember: serverId, userId, role/status

Wallet: serverMemberId, balance

Transaction: user, server, type, amount, balanceBefore, balanceAfter, game, createdAt

7. Monetization Model

Free: basic games, economy, leaderboard and daily rewards.

Pro: all games, custom currency/rewards, advanced leaderboard, analytics, custom casino theme and tournaments.

Business: multiple servers, advanced analytics, custom branding, API access and priority support.

Validate pricing with actual server owners before locking prices. The strongest business model is to sell useful server-owner tools rather than monetizing users' gambling behavior.

8. 6-Week Development Plan

Week 1	Discord bot, database, wallet, /balance
Week 2	/daily, Coin Flip, Dice, transactions
Week 3	Slots, leaderboard, cooldowns, anti-exploit
Week 4	Admin settings, dashboard, Discord OAuth
Week 5	Deployment, testing, beta launch
Week 6+	Blackjack, Roulette, tournaments, premium features

9. Launch Checklist

- Bot works reliably in a private test server.
- Slash commands are registered correctly.
- All wallet changes are recorded as transactions.
- Concurrent bets cannot create or duplicate coins.
- Negative, zero, oversized and invalid bets are rejected.
- Daily reward cannot be claimed repeatedly.
- Bot token and database credentials are stored only in environment variables.
- .env and other secrets are excluded from Git.
- Production database migrations have been tested.
- Discord installation uses only required permissions.
- At least a few real server admins have tested the product.
- Real-money features are not enabled without separate legal and policy review.

10. Final Product Architecture

Discord → discord.js → Commands → Economy / Games / Leaderboard → Prisma → PostgreSQL → Web Dashboard → Discord OAuth2

Recommended first milestone: build only **/balance + /daily + /coinflip** with a PostgreSQL wallet and transaction system. Once that is stable, add Dice and Slots.